Final Capstone Reflection

Medical Alert Interrupt Latency Demonstration

For my final Real Time systems capstone, I chose to build upon and refine my medical alert interrupt latency application. The system uses a simulated patient call button connected to an ESP32 in Wokwi. A falling edge interrupt records the event, produces a GPIO pulse for timing, and signals two FreeRTOS bottom half tasks. One is signaled through a binary semaphore and one through a direct task notification. I then compared the two signaling paths under idle conditions, under a controlled background workload, and with the ISR exit yield intentionally removed. The project brought together interrupts design, task priorities, timing measurements, fault injection, and technical communication in a way that can be demonstrated and documented.

I. What Was Harder Than Expected

The most difficult part was not getting interrupts to work but rather it was determining what the measurements truly meant. At first, it was easy to treat every timing value as though it represented the same quantity. In reality, the project included several different measurements. There was GPIO interrupt response time, ISR to bottom half wake latency, and the maximum observed duration of the periodic load tasks. Learning to separate those measurements was important as each one describes a different part of the system and only after analyzing them together can the system's operations be analyzed more thoroughly.

The background workloads also required a lot more adjustment than I expected. The original task sizes repeatedly triggered the watchdog timer in Wokwi, so I reduced the iteration and buffer sizes while preserving each of the four tasks, their priority order, and their heartbeat monitoring. This taught me that a demonstration environment has practical limits and that a stable, controlled test is more useful than an unrealistic test that cannot complete.

Another unexpected challenge was interpreting the fault injection. I predicted that removing `portYIELD_FROM_ISR()` would cause a large increase in latency. The measured increase under load was real but smaller than expected, especially for the direct task notification. That result was still valuable because it showed that fault injection is not only about proving a prediction, but it is also about comparing the prediction against evidence and explaining why the observed system behavior may be more complicated.

II. What I Would Do Differently

If I restarted the project, I would define the measurement plan more before collecting data. I would write down exactly which event begins and ends each timing interval. This would make the results easier to organize and would prevent inconsistent labels for photos, tables, and written analysis.

I would also automate more of the testing. Instead of manually pressing the button one hundred times for each configuration, I would add an event generator for repeatable test modes. That would reduce variation between runs and make it easier to collect minimum, maximum, and average latency values. I would also add more test types with one blocking object on and the other off and then the inverse.

Finally, I would use a method that can isolate CPU execution time or capture a detailed trace so that execution, ready, blocked, and preempted states can be distinguished directly. This would greatly benefit deeper analysis to see why latencies do or do not increase.

III. Most Valuable Lesson Learned

The most valuable lesson from this project was that real time performance depends on the complete execution path and not just whether the code is logically correct. The ISR itself remained short and used ISR safe APIs, but the response observed by the application still depended on task priorities, processor contention, the signaling primitive, and whether the scheduler was asked to switch immediately after the interrupt.

The project also made the difference between a binary semaphore and a direct task notification more concrete. The direct task notification produced the lower measured worst-case latency in the normal idle and loaded tests. More importantly, I now understand why the design choice should be connected to the communication contract. A direct notification is efficient for signaling one specific task while a binary semaphore is a separate synchronization object and can operate with multiple event tasks.

Beyond the technical implementation, I learned through this capstone but also through my entire course how to more effectively communicate engineering evidence. These forms of communication include various diagrams, task tables, screenshots, and or timing numbers and their usefulness is limited to the effectiveness of their explanations.

IV. Conclusion

Overall, this capstone strengthened both my technical understanding and my ability to present an embedded system professionally. I completed a working FreeRTOS demonstration, compared two ISR to task communication methods, observed effects of scheduled processor load, and tested a deliberate fault. The final result is not a production ready system, but it is a useful demonstration of the design and measurement decisions that matter in real time embedded systems. The project gave me a stronger foundation for future work involving interrupt driven firmware, timing analysis, and safety conscious designs.